

Reconstructing clade-founder genomes for Pango lineages

A step-by-step walkthrough of the Snakemake pipeline: what each step does, why it is there, and what it produces. Written for readers who have never used Snakemake.

Pipelines: `Snakefile_pango_extract` and `Snakefile_pango_founders` · Script: `scripts/5b_pango_founder_fasta.py`

1 What this pipeline is for

SARS-CoV-2 evolves as it spreads: each transmission can introduce mutations, and the genomes sampled worldwide form a giant family tree. Public databases distribute this tree in a compact form called a **mutation-annotated tree** (MAT) — a phylogeny of millions of genomes with the mutations placed on the branches where they arose.

Related genomes are grouped into named **lineages**. Two naming systems are in common use: **Nextstrain clades** (coarse labels such as `20B` or `24E`) and **Pango lineages** (fine-grained labels such as `AD.2` or `KP.3.1.1`). One Nextstrain clade can contain hundreds of Pango lineages. Section 6 shows exactly why that difference matters for our analysis.

To study how mutations affect viral fitness, we first need each lineage's **founder genome** — the reconstructed ancestral sequence at the point the lineage began. Every later mutation in that lineage is then measured relative to this founder. If the founder is wrong or too coarse, every downstream count is off. So the two jobs of this pipeline are:

1. **Extract** the part of the global tree belonging to a chosen Pango lineage, and pull out its mutations and sample paths.
2. **Reconstruct the founder genome** (a FASTA sequence) for that lineage, plus the list of substitutions that define it.

Both are done *per Pango lineage*, and the lineages of interest are simply listed in a configuration file.

2 Snakemake in five minutes

Snakemake is a workflow manager. Instead of running commands by hand in the right order, you declare a set of **rules**, and Snakemake works out the order for you and runs only what is needed.

Each rule declares three things:

- **input** — the files it needs before it can run;
- **output** — the files it produces;
- **a command or script** — how to turn input into output.

Snakemake reads the outputs you asked for and chains rules backwards: if rule B needs a file that rule A produces, it runs A first. The resulting graph of "which rule feeds which" is called the **DAG** (directed acyclic graph). Two features do most of the work:

- **Wildcards.** A pattern like `{pango}.fa` lets one rule stand in for many files. Ask for `KP.3.1.1.fa` and Snakemake fills `{pango}=KP.3.1.1`; ask for three lineages and it runs the same rule three times.
- **Only-what's-needed.** If an output already exists and its inputs haven't changed, Snakemake skips it. Re-running after a change rebuilds just the affected part.

Two commands you will use constantly:

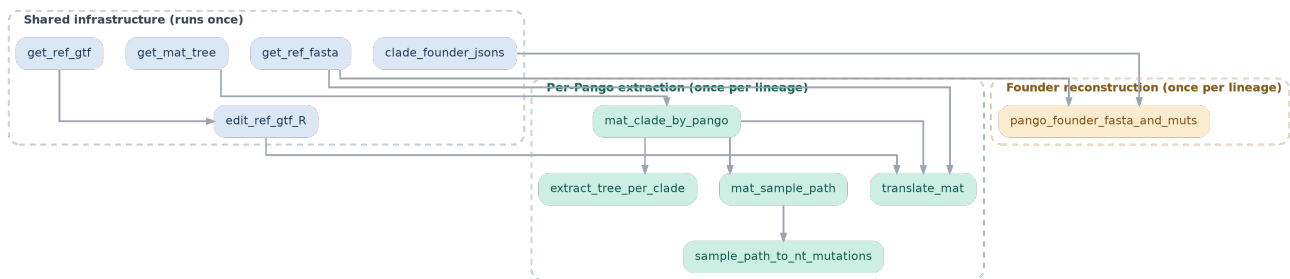
```
# Dry run: show what WOULD run, without running anything
snakemake -s Snakefile_pango_extract --configfile config_pango_extract.yaml -n

# Real run, using up to 4 CPU cores
snakemake -s Snakefile_pango_extract --configfile config_pango_extract.yaml --cores 4
```

A dry run (`-n`) is the safe way to check the plan before committing compute. It is how every rule in this document was validated.

3 The pipeline at a glance

The work splits into three coloured groups. The **infrastructure** rules (blue) download the tree and reference and run once. The **extraction** rules (green) and the **founder** rule (amber) run once per Pango lineage. Arrows mean "produces a file that the next rule needs".



Rulegraph for `Snakefile_pango_extract`. Boxes are rules, not files; the same shape underlies `Snakefile_pango_founders`, which keeps only the blue reference/JSON rules and the amber founder rule.

There are two Snakefiles because there are two natural stopping points:

File	Does	Needs the 5 GB MAT?
<code>Snakefile_pango_founders</code>	Founder FASTA + defining substitutions only.	No — reference + JSON only.
<code>Snakefile_pango_extract</code>	Everything: tree extraction, mutation tables, sample paths, <i>and</i> the founder.	Yes.

If you only need founder sequences, run the first — it is far lighter. If you need the tree and mutation lists too, run the second. The rest of this document walks through the fuller `Snakefile_pango_extract`; the founder-only pipeline is a strict subset of it.

4 The steps, one by one

For each rule: **what** it does, **why** we need it, its **input** and **output**, and what a correct output looks like.

4.1 Shared infrastructure RUNS ONCE

get_mat_tree

What / why. Downloads the pre-built global mutation-annotated tree from UCSC. This is the phylogeny of millions of SARS-CoV-2 genomes with mutations on its branches; every extraction step reads from it.

```
curl {url} > results_{mat}/mat/mat_tree.pb.gz
```

Input

A URL (from config `mat_trees`).

Output

`results_{mat}/mat/mat_tree.pb.gz`

Looks like

A compressed protobuf file, several gigabytes. Not human-readable — it is read by `matUtils`.

get_ref_fasta

What / why. Downloads the Wuhan-Hu-1 reference genome and unzips it. This fixes the coordinate system: every site number and every mutation in the whole pipeline is defined relative to this sequence.

```
wget -O - {url} | gunzip -c > results_{mat}/ref/ref.fa
```

Output

`results_{mat}/ref/ref.fa`

Looks like

A single-record FASTA, header `>NC_045512v2`, sequence length **29,903** nt.

get_ref_gtf

What / why. Downloads the gene annotation (where each gene starts and ends). Needed so nucleotide mutations can later be read as amino-acid changes ("which gene, which codon").

Output

`results_{mat}/ref/original_ref.gtf`

Looks like

A tab-separated annotation table, one row per gene/feature.

edit_ref_gtf_R

What / why. Runs a small R script to add `ORF9b` (positions 28284–28577) to the annotation. `ORF9b` overlaps the N gene and is missing from the stock file, but it is biologically important, so we insert it.

```
Rscript scripts/l_ref_gtf_minimal.R original_ref.gtf edited_ref_R.gtf edits.json
```

Input

`original_ref.gtf` + the edits from config `add_to_ref_gtf`.

Output

`results_{mat}/ref/edited_ref_R.gtf`

Looks like

The same GTF with one extra feature row for `ORF9b`.

clade_founder_jsons

What / why. Downloads two reference files of founder genotypes: the Neher `clade_gts.json` (keyed by Nextstrain clade) and the Roemer `pango-consensus-sequences_summary.json` (keyed by Pango lineage). The Pango pipeline uses **only the Roemer JSON**, because it is already indexed by the labels we care about.

Output

`clade_founders_neher.json`, `clade_founders_roemer.json`

Looks like

Two JSON files. The Roemer one contains ~5,000 Pango lineages, each with its list of nucleotide substitutions relative to the reference.

4.2 Per-Pango extraction ONCE PER LINEAGE

mat_clade_by_pango THE KEY STEP

What / why. Cuts the one chosen Pango lineage out of the global tree, using the tree's own Pango annotation. This is where Pango precision enters: `-c "KP.3.1.1"` pulls exactly that lineage's subtree, rather than a coarse Nextstrain bucket that lumps many lineages together (Section 6).

```
matUtils extract -i mat_tree.pb.gz -c "{pango}" -0 -o {pango}.pb
```

Input

The global MAT.

Output

`results_{mat}/pango_extract/{pango}.pb`

Looks like

A small MAT protobuf containing only that lineage's subtree. Much smaller than the input.

extract_tree_per_clade

What / why. Writes the extracted subtree as a plain **Newick** tree — the standard text format that tree-plotting and phylogenetics tools read.

```
matUtils extract -i {pango}.pb -t {pango}_tree.nwk
```

Output

```
{pango}_tree.nwk
```

Looks like

A text file of nested parentheses with tip names and branch lengths.

translate_mat

What / why. Reads the nucleotide mutations on the subtree and translates them into **amino-acid changes** using the reference and the edited GTF — telling you which protein and codon each mutation hits.

```
matUtils summary -i {pango}.pb -g edited_ref_R.gtf -f ref.fa -t {pango}_mutations.tsv
```

Input

The subtree + reference FASTA + edited GTF.

Output

```
{pango}_mutations.tsv
```

Looks like

A tab-separated table of mutations with their gene and amino-acid consequence.

mat_sample_path

What / why. For every sample (tip) in the clade, lists the ordered mutations from the tree root down to that tip. Unlike the translation step, this **keeps noncoding sites**, so it is the complete raw material for counting nucleotide mutations.

```
matUtils extract -i {pango}.pb --sample-paths {pango}_sample_paths.tsv
```

Output

```
{pango}_sample_paths.tsv
```

Looks like

One row per sample, each listing the mutation path (e.g. C241T > C3037T > ...).

sample_path_to_nt_mutations

What / why. A Python script that parses those path strings into a tidy table — one row per nucleotide mutation, with its site, the base it changed from and to, and where on the tree it occurred. This is the clean, analysis-ready mutation list.

```
script: scripts/4_sample_path_to_nt_mutations.py
```

Input

```
{pango}_sample_paths.tsv
```

Output

```
{pango}_nt_mutations.csv
```

Looks like

A CSV of nucleotide mutations, ready for counting and fitness analysis.

4.3 Founder reconstruction

ONCE PER LINEAGE

pango_founder_fasta_and_muts

What / why. Reconstructs the lineage's **founder genome**: start from the reference and apply exactly the substitutions that define the lineage (skipping indels). This founder is the baseline every within-lineage mutation is measured against. Because it reads the Roemer JSON *directly by Pango name*, no Nextstrain translation is involved.

```
script: scripts/5b_pango_founder_fasta.py
```

Input

Roemer JSON + reference FASTA + the wildcard `{pango}` .

Output

`{pango}.fa` (founder genome) and `{pango}_ref_to_founder_muts.csv` (defining substitutions).

Looks like

A single-record FASTA (header = the Pango name, length 29,903 nt) and a CSV of the substitutions.

5 Inside the founder script

This is the one script written specially for the Pango workflow, so it is worth understanding. It does four things:

1. **Read the reference** into memory (enforcing that it is a single record).
2. **Look up the lineage** in the Roemer JSON by its Pango name. If the name is not present, it stops and lists nearby names, so typos are obvious.
3. **Apply each substitution.** A substitution like `C241T` means "at site 241 the reference has `C` ; change it to `T`". The script checks that the reference really does have `C` there before changing it — a mismatch would signal a coordinate or reference-version error, and by default it stops (`founder_strict_ref_check: true`). Deletions are listed in the JSON but deliberately **not applied**, giving a "no-indels" founder.
4. **Write outputs:** the founder FASTA, and a CSV with columns `pango`, `nextstrain_clade`, `nt_site`, `ref_nt`, `founder_nt`, `nt_mutation`. The `nextstrain_clade` value is copied from the JSON purely as a label — it never drives the reconstruction.

Why the strict check matters. Substitution positions are 1-based and relative to Wuhan-Hu-1. If the wrong reference were supplied, hundreds of positions would disagree, and the script would refuse rather than silently build a corrupt founder. This is the pipeline's built-in guard against the most common class of error.

6 Why Pango, not Nextstrain

This is the whole reason the pipeline was rebuilt around Pango. A Nextstrain clade is a coarse bucket; a Pango lineage is precise. The consequences are concrete and measurable. Take Nextstrain clade `20B` and the Pango lineage `AD.2` inside it:

Question	Nextstrain 20B	Pango AD.2
----------	----------------	------------

How many lineages does the label cover?	480 Pango lineages lumped together	Exactly one
Founder substitutions	7 (B.1.1-level ancestor)	18 (true AD.2 ancestor)
Sample set extracted	All samples of all 480 lineages	Just the AD.2 subtree

The founder gap is the crux. The coarse 20B founder is missing the 11 substitutions that actually define AD.2:

A1163T T3256C C5622T G14202T G19542T C19718T C22388T C26060T G29227T C29466T
A29771G

Measured against the coarse founder, those 11 fixed differences would be miscounted as mutations *arising within the clade*, rather than as part of the ancestral background. That error flows straight into mutation counts, expected-versus-observed comparisons, and therefore the fitness and epistasis estimates. The Pango founder removes it.

The honest framing. The old and new results are not a like-for-like comparison to be reconciled mutation-by-mutation — the old 20B "clade" is a superset of 480 Pango lineages. The correct statement is that **Pango resolves what Nextstrain aggregates**: here, a 480-to-1 collapse with an 11-substitution founder error for AD.2 specifically.

7 How to run it

Prerequisites. Run from the repository root, so the `scripts/` folder is found. You need Snakemake, plus `matUtils` (from USHER) on your PATH for the extraction pipeline.

Step 1 — choose your lineages. Edit the config and list the Pango lineages you want:

```
pango_clades:
- KP.3.1.1
- XEC
- LP.8.1
```

Step 2 — check the plan (dry run):

```
snakemake -s Snakefile_pango_extract --configfile config_pango_extract.yaml -n
```

Step 3 — run it:

```
snakemake -s Snakefile_pango_extract --configfile config_pango_extract.yaml --cores 4
```

For founder sequences only (no tree, no 5 GB download), swap in the lighter pipeline:

```
snakemake -s Snakefile_pango_founders --cores 1
```

8 Expected outputs

Per Pango lineage, under `results_{mat}/pango_extract/`:

File	Produced by	Contents
<code>{pango}.pb</code>	<code>mat_clade_by_pango</code>	Extracted MAT subtree for the lineage.
<code>{pango}_tree.nwk</code>	<code>extract_tree_per_clade</code>	Newick tree of the lineage.
<code>{pango}_mutations.tsv</code>	<code>translate_mat</code>	Mutations with amino-acid consequences.
<code>{pango}_sample_paths.tsv</code>	<code>mat_sample_path</code>	Per-sample root-to-tip mutation paths (incl. noncoding).
<code>{pango}_nt_mutations.csv</code>	<code>sample_path_to_nt_mutations</code>	Tidy nucleotide-mutation table.
<code>{pango}.fa</code>	<code>pango_founder_fasta_and_muts</code>	Reconstructed founder genome (29,903 nt).
<code>{pango}_ref_to_founder_muts.csv</code>	<code>pango_founder_fasta_and_muts</code>	Substitutions defining the founder.

9 Sanity-checking a founder

Because the founder underpins everything, it is worth spot-checking. A quick, decisive test is to confirm the reconstructed substitutions match the Roemer consensus and that the clade backbone makes sense. For AD.2, for example, the founder should carry:

- the 20A backbone `C241T`, `C3037T`, `C14408T`, `A23403G` (the last is Spike D614G);
- the B.1.1 trinucleotide at 28881–28883 (`G28881A`, `G28882A`, `G28883C`), which is what places it in Nextstrain 20B;
- 18 substitutions in total, with no deletions applied.

If those hold, both the sequence and its clade assignment are internally consistent. A single command — comparing your `{pango}_ref_to_founder_muts.csv` against the Roemer JSON entry — is enough to confirm a lineage before trusting its downstream numbers.

10 Glossary

Term	Meaning
MAT	Mutation-annotated tree: a phylogeny with mutations placed on its branches, stored as a <code>.pb</code> protobuf.
USHER / matUtils	The toolkit that builds and queries MATs. <code>matUtils extract</code> and <code>summary</code> do the tree cutting and translation here.
Pango lineage	Fine-grained lineage label (e.g. <code>AD.2</code> , <code>KP.3.1.1</code>).
Nextstrain clade	Coarser label (e.g. <code>20B</code>); one clade can span hundreds of Pango lineages.
Founder genome	Reconstructed ancestral sequence for a lineage; the baseline for counting later mutations.
FASTA	Plain-text sequence format: a header line starting <code>></code> then the sequence.
Newick	Text format for trees, using nested parentheses.
	The 29,903-nt genome that fixes all site coordinates.

Reference (Wuhan-Hu-1)	
Rule / DAG / wildcard	Snakemake's building block; the graph of rule dependencies; and the pattern (<code>{pango}</code>) that lets one rule serve many files.